

Task 05 — Build & Test Basic Multi-Microservice Architecture

Step 1: Start User Service

Run your **User Service Spring Boot application**.

This service is responsible for user-related operations such as:

- Creating users
- Getting users
- Updating users
- Deleting users

Open your User Service project in **IntelliJ IDEA / Eclipse / STS**.

@SpringBootApplication

```
public class UserServiceApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(UserServiceApplication.class, args);  
    }  
}
```

Run it

use the terminal:

```
mvn spring-boot:run
```

Expected output

```
Started UserServiceApplication
```

This means your User Service is running.

Step 2: Start Order Service

Run your **Order Service Spring Boot application** separately.

This service is responsible for:

- Creating orders
- Getting orders
- Updating orders
- Deleting orders

Open your **Order Service project** in another IntelliJ window or terminal.

@SpringBootApplication

```
public class OrderServiceApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(OrderServiceApplication.class, args);  
    }  
}
```

Run it:

```
mvn spring-boot:run
```

Expected output

```
Started OrderServiceApplication
```

Now both services should be running.

```
User Service → Running
```

```
Order Service → Running
```

Step 3: Verify both health endpoints

We need to check whether both applications are **alive and working**.

Spring Boot provides a health-check API through **Actuator**.

Endpoint

```
GET /actuator/health
```

Example

If User Service runs on port 8081:

```
GET http://localhost:8081/actuator/health
```

If Order Service runs on port 8082:

```
GET http://localhost:8082/actuator/health
```

Expected response

```
{  
  "status": "UP"  
}
```

The Spring Boot application is running and its health check is successful.

If you get this error

404 Not Found

Check whether Actuator is added to your project.

For Maven:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Then restart the application.

Step 4: Create users through User Service

We must create users **using the User Service API**.

We should **not directly insert users into the Order Service database**.

Example API

Assume your User Service has this controller:

```
@RestController
@RequestMapping("/users")
public class UserController {
    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }
}
```

Test in Postman

Method: POST

URL: http://localhost:8081/users

Body → raw → JSON:

```
{  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Expected response

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

The user is saved in the **User Database**.

Step 5: Create orders through Order Service

We must create orders **using the Order Service API**.

Example controller

```
@RestController  
@RequestMapping("/orders")  
public class OrderController {  
    @PostMapping  
    public Order createOrder(@RequestBody Order order) {  
        return orderService.createOrder(order);  
    }  
}
```

Test in Postman

Method: POST

URL: http://localhost:8082/orders

Body:

```
{  
  "productName": "Laptop",  
  "quantity": 1,  
  "userId": 1  
}
```

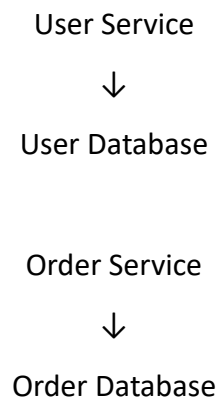
Expected response

```
{  
  "id": 1,  
  "productName": "Laptop",  
  "quantity": 1,  
  "userId": 1  
}
```

The order is saved in the **Order Database**.

Step 6: Verify that each service uses its own database

Each microservice should have **its own database**.

**Example**

User Service configuration:

```
spring.datasource.url=jdbc:mysql://localhost:3306/user_db
```

Order Service configuration:

```
spring.datasource.url=jdbc:mysql://localhost:3306/order_db
```

Open MySQL Workbench or your database tool.

Check: SHOW DATABASES;

We see like:

```
user_db
```

```
order_db
```

Then check the tables:

```
USE user_db;
```

```
SELECT * FROM users;
```

You should see the user you created.

Now:

```
USE order_db;
```

```
SELECT * FROM orders;
```

You should see the order you created.

User Service owns user data, and Order Service owns order data.

The Order Service should not directly access the User Service database.

Step 7: Test all APIs using Postman

You need to test **all the APIs we created**.

User Service

Method	URL	Purpose
POST	/users	Create user
GET	/users	Get all users
GET	/users/{id}	Get user by ID
PUT	/users/{id}	Update user
DELETE	/users/{id}	Delete user

Order Service

Method	URL	Purpose
POST	/orders	Create order
GET	/orders	Get all orders
GET	/orders/{id}	Get order by ID
PUT	/orders/{id}	Update order
DELETE	/orders/{id}	Delete order

Example: Get all users

GET http://localhost:8081/users

Expected:

```
[
  {
    "id": 1,
    "name": "Ravi",
    "email": "ravi@gmail.com"
  }
]
```

Example: Get all orders

GET http://localhost:8082/orders

Expected:

```
[
  {
    "id": 1,
    "productName": "Laptop",
    "quantity": 1,
    "userId": 1
  }
]
```

Step 8: Test invalid requests

Check what happens when the user sends **wrong or incomplete data**.

Postman → Select your API → Send invalid data.

Example 1: Missing required field

```
{  
  "name": "Ravi"  
}
```

If email is required, the application should reject the request.

Example 2: Invalid email

```
{  
  "name": "Ravi",  
  "email": "ravi123"  
}
```

Example 3: Empty name

```
{  
  "name": "",  
  "email": "ravi@gmail.com"  
}
```

Expected output:

```
{  
  "name": "Name is required",  
  "email": "Invalid email"  
}
```

Expected HTTP status: 400 Bad Request

Step 9: Test database failure scenarios

Check how your application behaves when the **database is unavailable**.

1. Start your User Service.
2. Stop MySQL / PostgreSQL.
3. Open Postman.

4. Send a request to create or get a user.

Expected : Database connection failed

The application should return a proper error response instead of crashing unexpectedly.

Example response:

```
{  
  "message": "Database is unavailable"  
}
```

Expected HTTP status: 500 Internal Server Error

Database down → Application should handle the error properly.

Step 10: Stop User Service and observe how Order Service behaves

Check what happens when **Order Service depends on User Service**, but User Service is stopped.

architecture

Client

v

Order Service

| Calls User Service

User Service

v

Database

Expected behavior:

Order Service

| Cannot connect

v

User Service (DOWN)

The Order Service should return a meaningful error.

Example response:

```
{  
  "message": "User Service is unavailable"  
}
```

Step 11: Check application logs

Open the **Console / Terminal** where we started your Spring Boot application.

Example log

Started UserServiceApplication

When an API is called:

Creating user

User saved successfully

When an error occurs:

Database connection failed

Step 12: Verify exception handling

Check whether your application handles errors properly instead of showing confusing errors.

In your Spring Boot project, usually inside:

src/main/java

└─ exception

Example exception class

```
public class UserNotFoundException extends RuntimeException {  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

Example global exception handler

@RestControllerAdvice

```
public class GlobalExceptionHandler {  
    @ExceptionHandler(UserNotFoundException.class)  
    public ResponseEntity<String> handleUserNotFound(  
        UserNotFoundException ex) {  
        return ResponseEntity  
            .status(HttpStatus.NOT_FOUND)  
            .body(ex.getMessage());  
    }  
}
```

```
}  
}
```

Example

If user ID 100 does not exist:

GET /users/100

Expected output:User not found

Expected status: 404 Not Found

Step 13: Verify validation

Check whether your application rejects invalid input according to the validation rules.

Do it in:

Usually in your **DTO class**.

Example DTO

```
public class UserRequest {  
    @NotBlank(message = "Name is required")  
    private String name;  
    @Email(message = "Invalid email")  
    @NotBlank(message = "Email is required")  
    private String email;  
    // Getters and setters  
}
```

Controller

@PostMapping

```
public ResponseEntity<User> createUser(  
    @Valid @RequestBody UserRequest request) {  
    return ResponseEntity.ok(userService.createUser(request));  
}
```

Test in Postman

```
{  
    "name": "",
```

```
"email": "wrong-email"
}
```

Expected output:

```
{
  "name": "Name is required",
  "email": "Invalid email"
}
```

Step 14: Verify environment configuration

Check whether your application is using the correct configuration for the environment.

do it in: src/main/resources/application.properties

Example User Service

```
spring.application.name=user-service
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/userdb
spring.datasource.username=root
spring.datasource.password=yourpassword
```

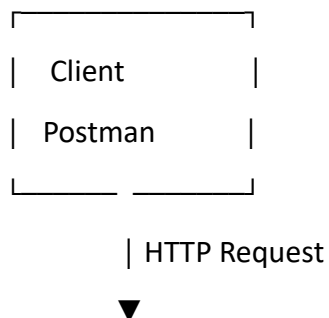
Example Order Service

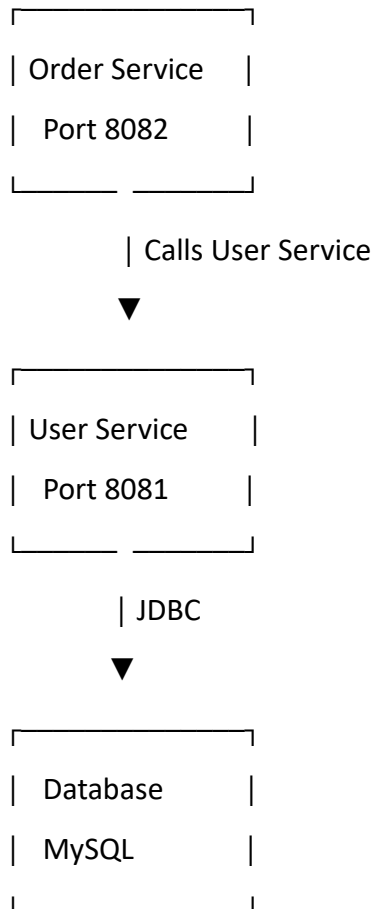
```
spring.application.name=order-service
server.port=8082
user.service.url=http://localhost:8081
```

Step 15: Create final architecture diagram

Draw the **complete architecture of your project** after implementation.

final architecture





Step 16: Create API documentation

Example API documentation

User Service API Documentation

1. Create User

Method: POST

URL: <http://localhost:8081/users>

Request Body:

```
{  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Success Response:

Status: 201 Created

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Information	Example
-------------	---------

API name	Create User
----------	-------------

HTTP method	POST
-------------	------

URL	/users
-----	--------

Request body	JSON
--------------	------

Success response 201 Created

Error response 400 Bad Request

Purpose Creates a new user

Step 17: Create a Postman collection containing all APIs

Open Postman.

Click **New → Collection**.

Give the name: Microservices API Collection

Add your User Service requests.

Add your Order Service requests.

Save all requests.

Example

POST http://localhost:8081/users

GET http://localhost:8081/users/1

POST http://localhost:8082/orders

GET http://localhost:8082/orders/1

Step 18: complete README

Microservices Project

1. Project Description

This project contains two Spring Boot microservices:

- User Service
- Order Service

The Order Service communicates with the User Service.

2. Technologies Used

- Java
- Spring Boot
- Spring Data JPA
- MySQL
- Maven
- Postman

3. Project Structure

user-service/

order-service/

4. How to Run

Start User Service

Run the User Service application.

Port: 8081

Start Order Service

Run the Order Service application.

Port: 8082

5. API Endpoints

User Service

POST /users

GET /users/{id}

Order Service

POST /orders

GET /orders/{id}

6. Testing

Use Postman to test all APIs.

7. Error Handling

The application handles:

- Invalid requests
- User not found
- Database failures
- Service unavailability